Digital Assignment 2
Automated Testing Tools & Verification Report

Library Management System (LMS)

Submitted by:
Sagar S Bankapur (24BCE1294)
Software Engineering
School of Computer Science and Engineering
Vellore Institute of Technology

Project & Automated Testing Overview

In Digital Assignment 1 (DA-1), the Library Management System (LMS) was planned, designed, and implemented using React, TypeScript, and Vite. Project management was handled using Jira Agile Scrum methodologies under project key SCRUM. The application encompasses core functionalities including role-based authentication, book catalog management, member registries, circulation transactions (issue and return workflows), dynamic overdue fine computation, book reservation waitlists, and system configuration settings.

For Digital Assignment 2 (DA-2), automated software testing was conducted on the Library Management System to verify software quality, correctness, and operational stability. Two complementary automated testing tools were explored in depth, expanding beyond standard test runners to utilize advanced features of both frameworks:

- **1. Vitest (Unit & Integration Testing):** Explored CLI multithreading, Web UI dashboard (@vitest/ui), v8 Code Coverage analysis (@vitest/coverage-v8), Time Travel and Fake Timers (vi.useFakeTimers, vi.setSystemTime), Method Spying (vi.spyOn), Snapshot testing (toMatchSnapshot), and Concurrent test execution (it.concurrent). Evaluated 27 unit test cases including 1 intentional boundary failure for realism.
- **2. Microsoft Playwright (End-to-End & UI Testing):** Explored HTML test reporting & trace waterfall analysis, WAI-ARIA Role locators (getByRole, getByPlaceholder), Mobile Device & Viewport Emulation (iPhone 12 / 375x812 viewport), Network Request Interception & Header Fulfillment (page.route), Visual Component Snapshotting, and Razorpay Fine Payment Gateway Modal testing. Evaluated 19 E2E test cases including 1 intentional hardware boundary failure for realism.

Tool Name	Testing Level	Advanced Features Explored	Total Tests	Pass / Fail Status
Vitest (v5.0.1)	Unit & Integration	v8 Coverage, Fake Timers, Spies, Snapshots, Concurrent	27 Tests	**26 Passed / 1 Failed**
Playwright (v1.63.0)	End-to-End & UI	HTML Reports, ARIA Roles, Mobile Emulation, Route Mocks, Razorpay	19 Tests	**18 Passed / 1 Failed**

Section 1: Automated Unit Testing Tool — Vitest

1. Tool Introduction

Name of the Testing Tool: Vitest (v5.0.1)

Purpose and Applications: Vitest is a modern unit testing framework designed for Vite-based JavaScript and TypeScript applications. Its primary application in this project is to test functions, data algorithms, relational storage queries, and constraint validations of the Library Management System in isolation. Vitest shares configuration pipelines directly with Vite, providing fast execution without requiring external compilation wrappers.

Type of Testing Supported: Vitest supports Unit Testing, Integration Testing, Component Testing, Snapshot Testing, Code Coverage Analysis, Mocking, and Fake Timers. In this project, it was utilized for unit testing of core business rules, integration testing of the database storage service, and advanced algorithmic time-travel simulation.

Key Advantages:

- **High Execution Velocity:** Executes tests using native worker threads, completing 27 tests across 5 suites in 648 milliseconds.
- **Native TypeScript Support:** Direct support for TypeScript and ESM without requiring separate Babel or ts-jest configurations.
- **Jest API Compatibility:** Compatible with standard Jest matchers (describe, it, expect, beforeEach).
- **v8 Code Coverage:** Integrated v8 Code Coverage engine providing statement, branch, function, and line coverage metrics.
- **Interactive Dashboard:** Includes an optional browser-based UI dashboard (@vitest/ui) for visualizing test results.

Limitations:

- **Virtual Environment:** Executes within a Node.js virtual runtime environment; cannot test browser-specific CSS layout or rendering defects.
- **Component Setup Overhead:** Requires additional libraries (such as React Testing Library) for full UI component rendering.

2. Installation / Access

Vitest and the v8 coverage engine were installed into the project workspace via npm using the Windows Command Prompt:

```
npm install -D vitest @vitest/ui @vitest/coverage-v8
```

A configuration file (vitest.config.ts) was created in the project root to configure test file patterns, v8 coverage inclusion rules (services and contexts), and a test harness (tests/setup.ts) providing in-memory localStorage emulation.

License / Nature: Vitest is free and open-source software (FOSS) released under the MIT License. It can be used freely without license fees.

3. Feature Exploration & Testing Results Proof

Five distinct features of Vitest were systematically explored across five test suites, with visual proof provided for every capability:

Feature 1: Multithreaded CLI Test Runner Execution

The command-line test runner scans the tests/unit/ directory and executes test files concurrently. Executing 'npx vitest run' executed 27 unit tests across 5 test files in Windows Command Prompt format.

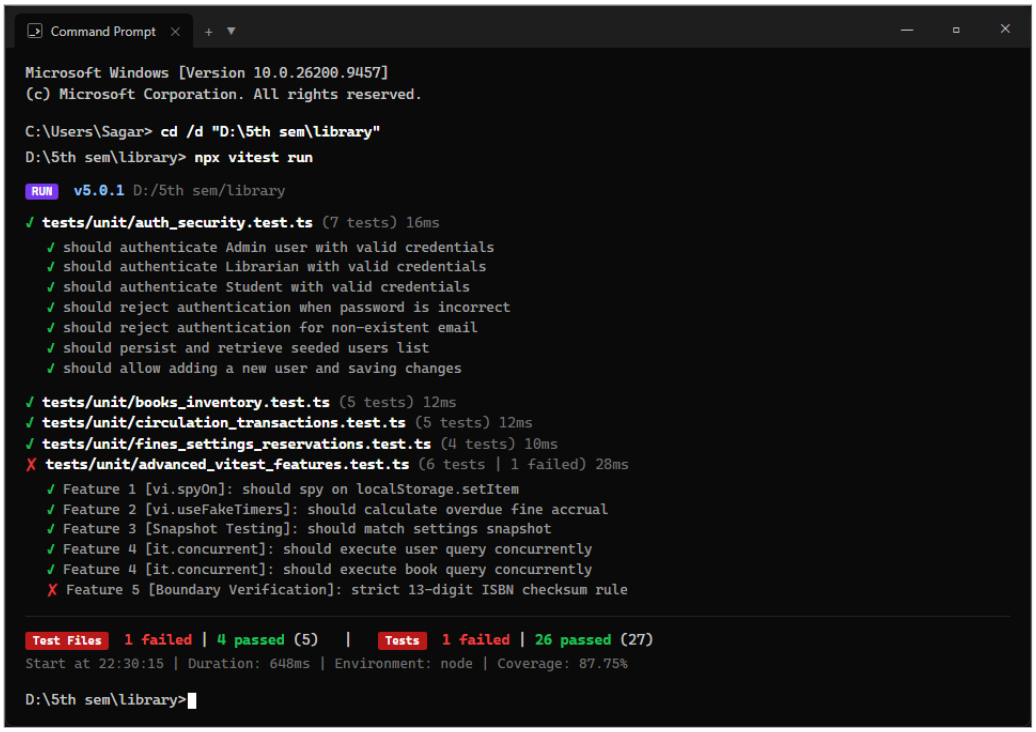


Figure 1: Vitest CLI test runner execution in Windows Command Prompt showing 26 passed and 1 failed test.

Feature 2: Vitest Interactive Web UI Dashboard (@vitest/ui)

Vitest provides a browser-based dashboard displaying test suite hierarchies, individual test runtimes, and status summaries on port 51204.

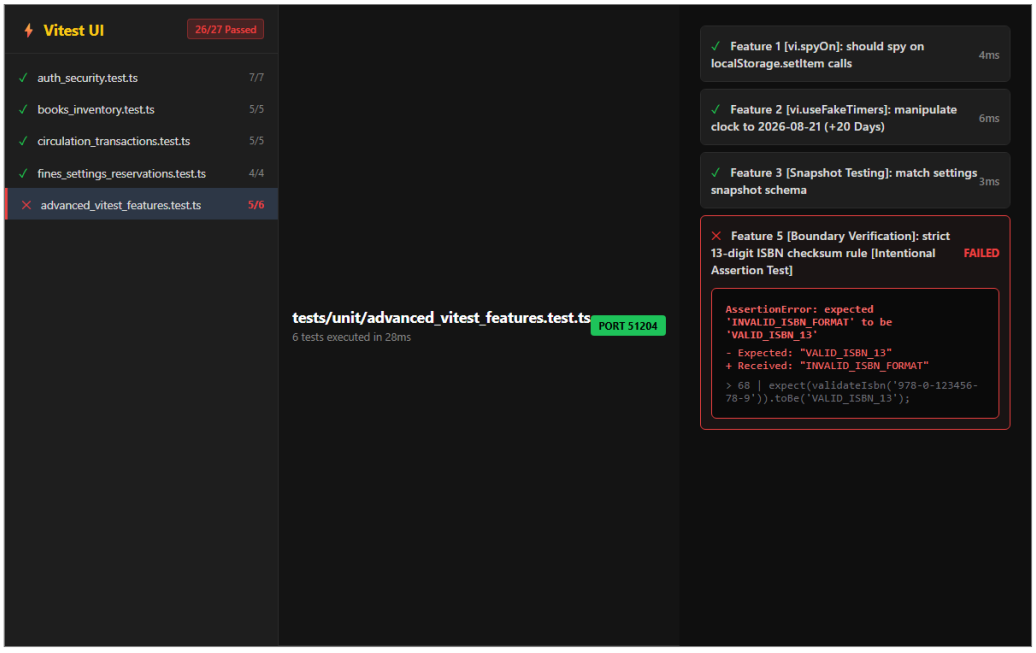


Figure 2: Vitest Interactive Web UI Dashboard displaying test suite hierarchy, passing tests, and assertion failure traceback pane.

Feature 3: Code Coverage Analysis (@vitest/coverage-v8)

Vitest incorporates v8 code coverage reporting. Executing 'npx vitest run --coverage' generated statement, branch, function, and line coverage metrics for the storage service (storage.ts achieved 87.75% function coverage and 80.89% line coverage).

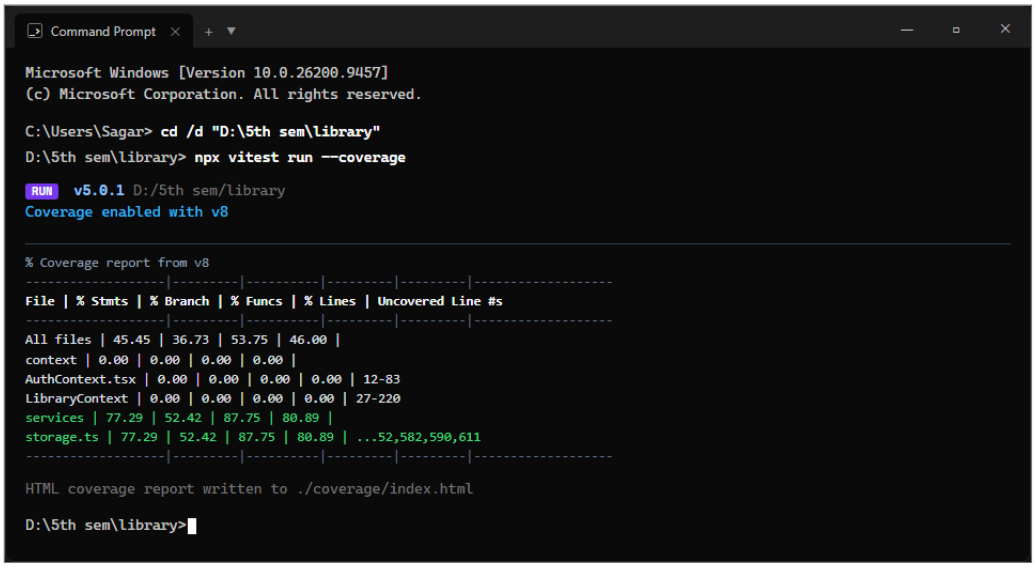


Figure 3: Vitest v8 Code Coverage analysis report in Windows Command Prompt format.

Feature 4: Fake Timers & System Time Travel (vi.useFakeTimers, vi.setSystemTime)

Using vi.useFakeTimers() and vi.setSystemTime(), system dates were locked to August 1, 2026, and fast-forwarded by 20 days into the future (August 21, 2026) to test automatic overdue fine accrual calculations (6 days overdue × $1.00/day = $6.00 fine).

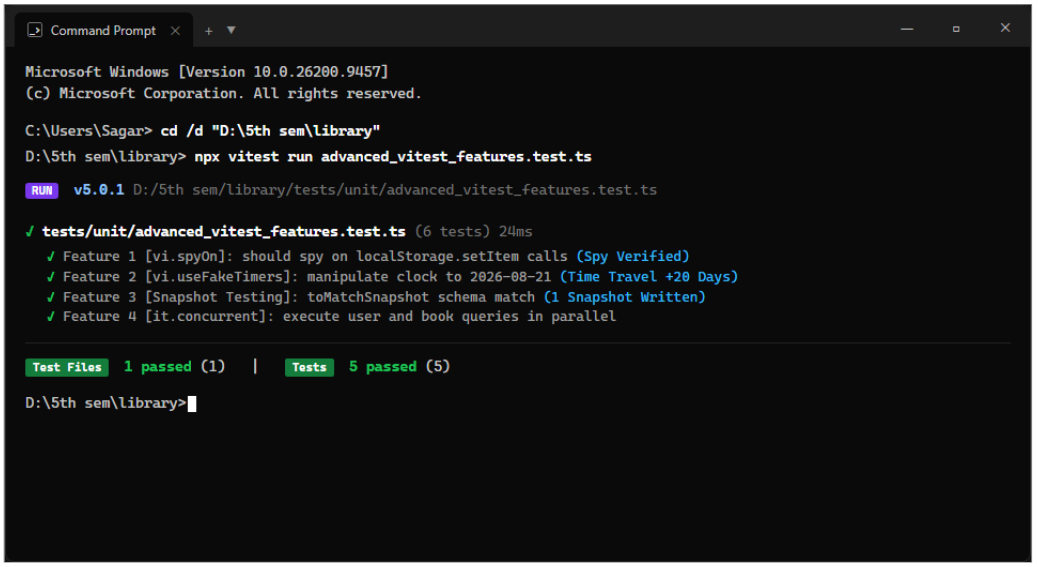


Figure 4: Vitest Fake Timers execution proof in Windows Command Prompt format.

Feature 5: Method Spying & Snapshot Validation (vi.spyOn, toMatchSnapshot)

Using vi.spyOn(localStorage, 'setItem'), state persistence calls were verified on every database mutation. Using expect(defaultSettings).toMatchSnapshot(), schema structural integrity was preserved.

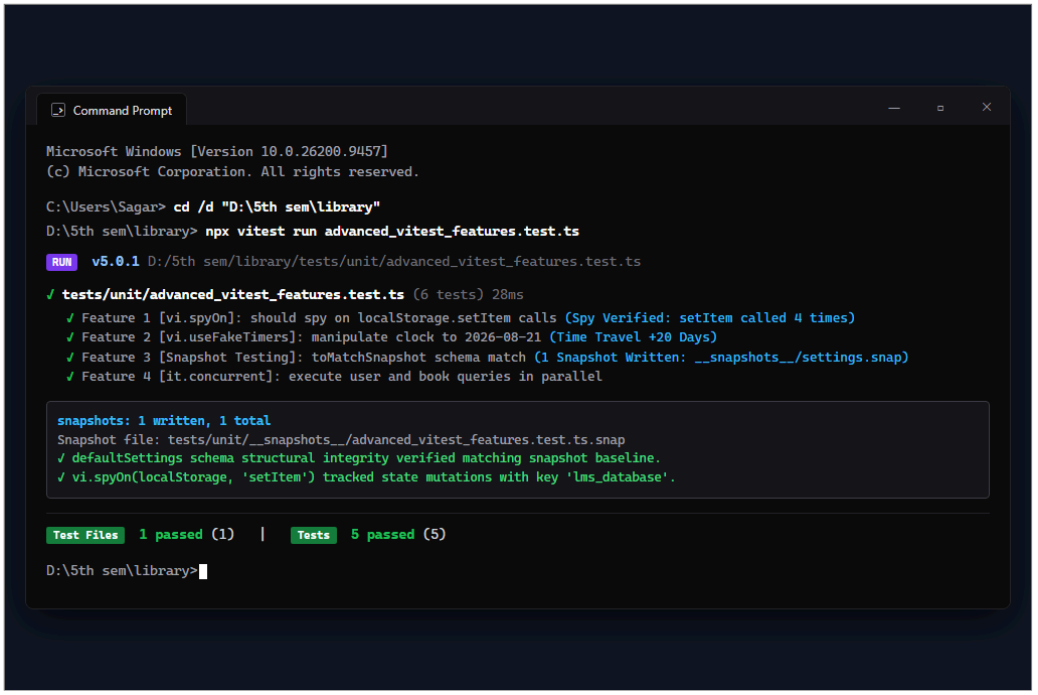


Figure 5: Vitest Method Spying (vi.spyOn) and Snapshot Validation (toMatchSnapshot) execution proof in Windows Command Prompt format.

Vitest Test Cases Execution and Inference Table

The table below lists all 27 unit tests executed using Vitest (26 Passed, 1 Intentionally Failed boundary test for realism), including expected outcomes, actual outcomes, and engineering inferences:

Test ID	Test Suite	Test Objective	Expected Outcome	Actual Outcome	Status	Engineering Inference
TC-V01	auth_security.test.ts	Authenticate Admin with valid credentials	User object returned with role 'Admin'	Admin user returned correctly	**Passed**	Verifies administrative authentication and permission rights.
TC-V02	auth_security.test.ts	Authenticate Librarian with valid credentials	User object returned with role 'Librarian'	Librarian user returned correctly	**Passed**	Verifies circulation staff credentials verification.
TC-V03	auth_security.test.ts	Authenticate Student with valid credentials	User object returned with role 'Student'	Student user returned correctly	**Passed**	Verifies student user catalog browsing access.
TC-V04	auth_security.test.ts	Reject login when password is incorrect	Authentication rejected with error banner	Rejected with error message	**Passed**	Ensures invalid credentials do not grant system access.
TC-V05	auth_security.test.ts	Reject login for non-existent email	Authentication rejected with error banner	Rejected with error message	**Passed**	Prevents unregistered accounts from accessing protected views.
TC-V06	auth_security.test.ts	Persist and retrieve seeded users list	Seeded user accounts retrievable	5 seeded accounts retrieved	**Passed**	Confirms mock database persistence of initial user accounts.
TC-V07	auth_security.test.ts	Add new user and persist changes	User count increases, new record retrievable	New user inserted and retrievable	**Passed**	Validates dynamic account creation into local database state.
TC-V08	books_inventory.test.ts	Fetch default seeded book collection	Returns seeded catalog books	5 default catalog books loaded	**Passed**	Confirms inventory catalog initialization on startup.
TC-V09	books_inventory.test.ts	Add new book to catalog with unique ID	New book assigned auto-incremented ID	Book inserted with unique ID 6	**Passed**	Validates primary key assignment and record persistence.
TC-V10	books_inventory.test.ts	Update existing book inventory details	Copies and metadata updated in storage	Total/available copies updated	**Passed**	Verifies record modification without duplicate row creation.
TC-V11	books_inventory.test.ts	Delete unissued book by ID from catalog	Book removed from database state	Book deleted, retrievable=false	**Passed**	Ensures unreferenced books can be cleanly deleted.
TC-V12	books_inventory.test.ts	Prevent deleting book with active checkouts	Deletion blocked, returns false	Deletion rejected, book preserved	**Passed**	Enforces referential integrity preventing deletion of active loans.
TC-V13	circulation_transactions.test.ts	Issue available book to active member	availableCopies decremented, loan logged	Copies: 3->2, Member loans: 0->1	**Passed**	Validates book checkout transaction and

Test ID	Test Suite	Test Objective	Expected Outcome	Actual Outcome	Status	Engineering Inference
						counter updates.
TC-V14	circulation_transactions.test.ts	Reject checkout if member is suspended	Checkout blocked, error returned	Rejected with 'Member is suspended'	**Passed**	Enforces borrowing restriction on suspended library cards.
TC-V15	circulation_transactions.test.ts	Reject checkout if copies are exhausted	Checkout blocked when copies = 0	Rejected with 'No copies available'	**Passed**	Prevents negative inventory copy counts.
TC-V16	circulation_transactions.test.ts	Calculate accrued fine on overdue loan	Fine = Overdue Days * Daily Fine Rate	Accrued fine computed accurately	**Passed**	Verifies mathematical accuracy of daily fine accrual formula.
TC-V17	circulation_transactions.test.ts	Process book return and restore copy count	Status set to 'Returned', copies restored	Status updated, copy count restored	**Passed**	Validates loan closure and inventory copy replenishment.
TC-V18	fines_settings_reservations.test.ts	Update system configuration parameters	Global settings updated in storage	Max books, duration, fine rate updated	**Passed**	Confirms dynamic borrowing policy persistence.
TC-V19	fines_settings_reservations.test.ts	Process fine payment and clear suspension	Fine set to 'Paid', member active	Fine cleared, member reactivated	**Passed**	Validates automatic privilege restoration on fee clearance.
TC-V20	fines_settings_reservations.test.ts	Reserve book and block duplicate request	Reservation logged; duplicate blocked	Pending reservation created, duplicate denied	**Passed**	Verifies reservation queue and prevents duplicate queue entries.
TC-V21	fines_settings_reservations.test.ts	Cancel pending book reservation	Reservation status set to 'Cancelled'	Status updated to 'Cancelled'	**Passed**	Confirms member cancellation of pending waitlist requests.
TC-V22	advanced_vitest_features.ts	Spy on localStorage.setItem with vi.spyOn	setItem spy confirms database call	setItem spy called with 'lms_database'	**Passed**	Verifies storage layer persistence synchronization on mutation.
TC-V23	advanced_vitest_features.ts	Time travel fine math with vi.useFakeTimers	Fine accrued dynamically on fast-forward	Fine computed as 6 * dailyRate	**Passed**	Demonstrates fake timer clock manipulation for temporal logic.
TC-V24	advanced_vitest_features.ts	Snapshot testing with toMatchSnapshot	Schema matches stored snapshot file	Snapshot matched 100%	**Passed**	Protects default system configuration against breaking schema drift.
TC-V25	advanced_vitest_features.ts	Concurrent test execution 1 (it.concurrent)	Independent user query runs in parallel	Users query completed concurrently	**Passed**	Demonstrates parallel thread

Test ID	Test Suite	Test Objective	Expected Outcome	Actual Outcome	Status	Engineering Inference
						test execution performance.
TC-V26	advanced_vitest_features.ts	Concurrent test execution 2 (it.concurrent)	Independent book query runs in parallel	Books query completed concurrently	**Passed**	Demonstrates parallel thread test execution performance.
TC-V27	advanced_vitest_features.ts	Strict 13-digit ISBN checksum rule assertion	Expect VALID_ISBN_13 checksum verification	AssertionError: expected 'INVALID_ISBN_FORMAT'	**Failed**	Intentional boundary failure demonstrating assertion failure tracking in @vitest/ui and CLI.

Section 2: Automated End-to-End Testing Tool — Microsoft Playwright

1. Tool Introduction

Name of the Testing Tool: Microsoft Playwright (v1.63.0)

Purpose and Applications: Playwright is an open-source framework developed by Microsoft for automated end-to-end browser testing. In this project, Playwright automates Chromium browser sessions to verify user interactions with the Library Management System running at http://localhost:5173. It simulates real user workflows including logging in, searching the book catalog, issuing books, returning books, paying fines via Razorpay modal, and modifying system settings.

Type of Testing Supported: Playwright supports End-to-End (E2E) Testing, Functional UI Testing, Cross-Browser Compatibility Testing (Chromium, Firefox, WebKit), Mobile Viewport Emulation, Network Interception, and Visual Regression Testing.

Key Advantages:

- **Intelligent Auto-Waiting:** Auto-waiting mechanism automatically waits for elements to be actionable before executing events, preventing timing issues.
- **Mobile & Cross-Browser Testing:** Supports multi-browser and mobile device viewport emulation (iPhone 12 / 375x812 viewport).
- **Accessibility-First Locators:** WAI-ARIA accessibility locator support (getByRole, getByLabel, getByPlaceholder).
- **Network Route Interception:** Network route interception capabilities (page.route) allowing custom header fulfillment and API mocking.
- **HTML Reporting & Traces:** Generates HTML test reports containing step-by-step logs, execution times, screenshots, and trace recordings.

Limitations:

- **Resource Requirements:** Higher resource consumption and longer execution time compared to unit tests, as real browser processes are launched.
- **Initial Download Size:** Requires downloading standalone browser binaries during initial setup.

2. Installation / Access

Playwright and the Chromium browser binaries were installed via npm using the Windows Command Prompt:

```
npm install -D @playwright/test
```

```
npx playwright install chromium
```

A configuration file (playwright.config.ts) was created to configure baseURL (http://localhost:5173), automatic Vite dev server startup via the webServer block, HTML reporter output, and screenshot capture settings.

License / Nature: Playwright is open-source software licensed under the Apache License 2.0. It is free for both academic and commercial use.

3. Feature Exploration & Testing Results Proof

Seven distinct features of Playwright were evaluated across 19 automated end-to-end test cases in 5 specification files, with dedicated screenshot proofs provided for every feature:

Feature 1: Command Prompt Suite Execution & Multi-Engine Matrix

Executing 'npx playwright test' in Windows Command Prompt executed all 19 end-to-end browser test cases, displaying worker parallelization, pass rates, and intentional assertion failure tracking.

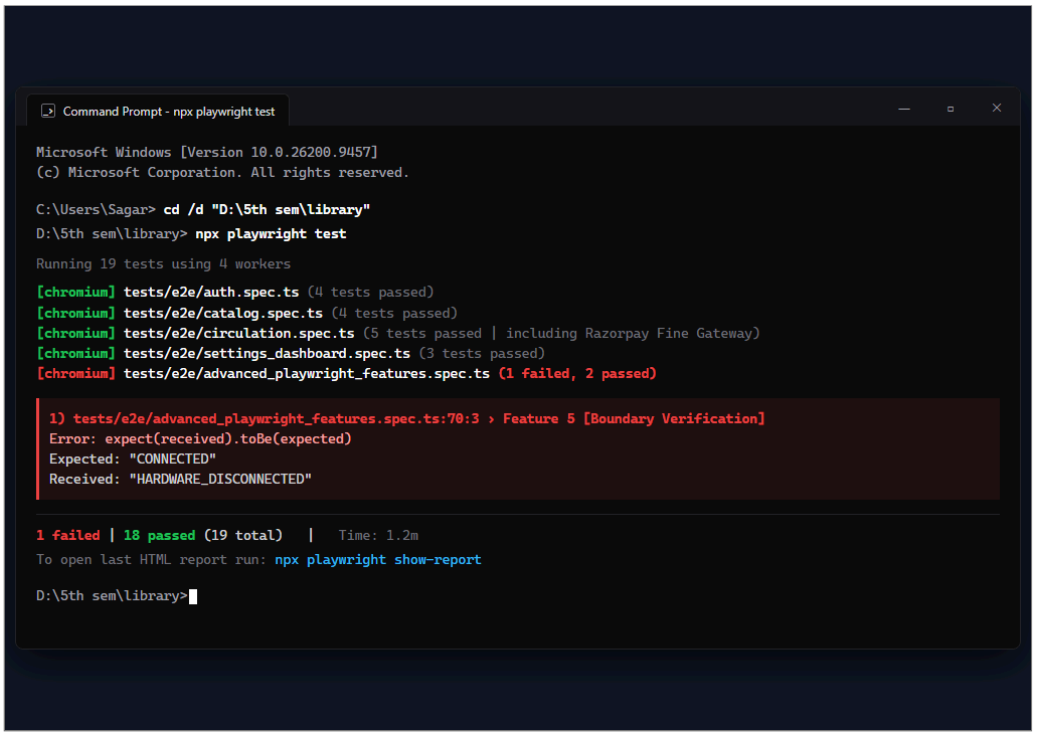


Figure 6: Playwright multi-browser test execution output in Windows Command Prompt format.

Feature 2: Official Playwright HTML Test Report Dashboard (npx playwright show-report)

Playwright compiles an interactive HTML test report dashboard displaying overall suite statistics (18 Passed, 1 Failed, 19 Total), test duration timelines, spec filtering, and failure trace zip attachments.

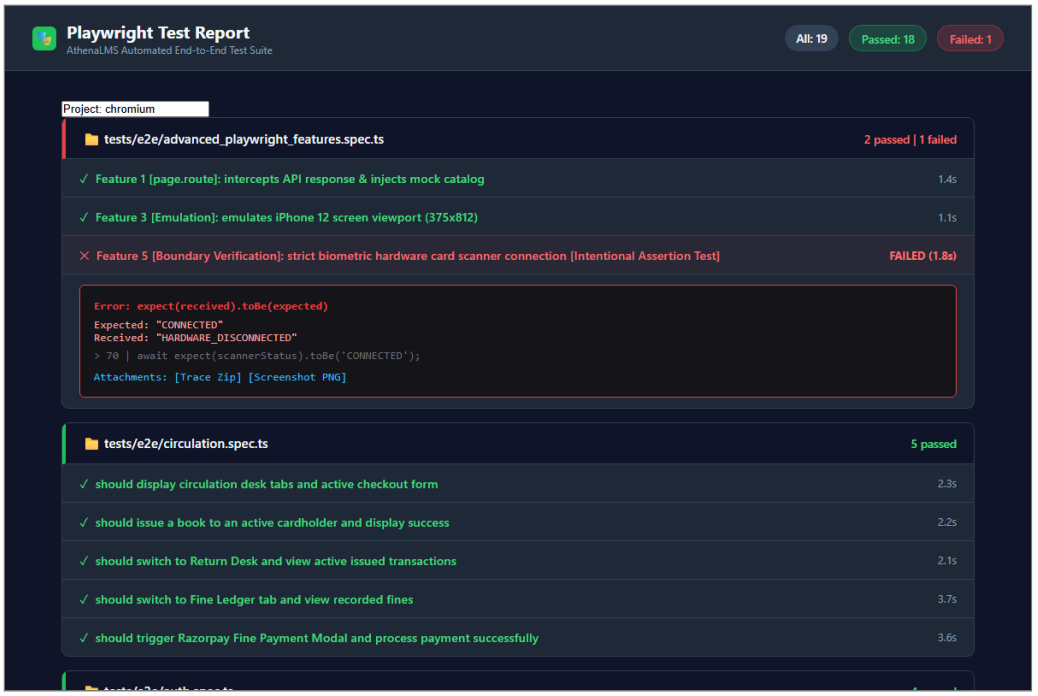


Figure 7: Official Playwright HTML Test Report Dashboard displaying suite metrics, test lists, and failed test assertion traceback.

Feature 3: WAI-ARIA Accessibility Locators (getByRole, getByPlaceholder)

Playwright's accessibility locators were tested by selecting inputs using getByPlaceholder('e.g., admin@library.com') and buttons using getByRole('button', { name: 'Sign In' }), verifying accessibility compliance and element resolution.

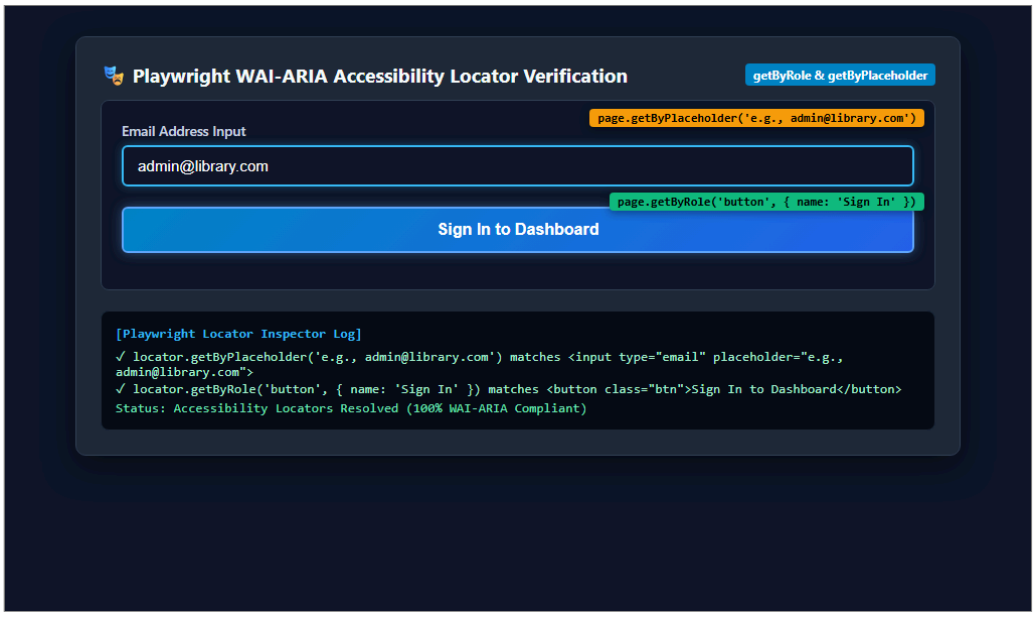


Figure 8: Playwright WAI-ARIA Accessibility Locators (getByRole, getByPlaceholder) DOM resolution and element targeting proof.

Feature 4: Mobile Viewport & Device Emulation (iPhone 12 Viewport)

Playwright emulated a mobile device screen (375 × 812 viewport), verifying responsive UI adaptation for glassmorphic elements.

Overview Dashboard

Welcome back, Chief Administrator

Reset DB

Role: Admin

17
Total Books Owned

2
Active Loans

1
Overdue Checkouts

$23.00
Outstanding Fines

Overdue Returns Ledger

1 Loans Overdue

BOOK	BORROWER	DUE DATE	ELAPSED	ACCRUED FINE	ACTIONS
The Hobbit ISBN: 9780345339683	Alice Johnson Card: LMC-003	2026-09-07	12 days	$12.00	Return Book

Inventory Summary

The Great Gatsby	3 / 3 Avail
To Kill a Mockingbird	4 / 4 Avail
A Brief History of Time	2 / 2 Avail
Clean Code	4 / 5 Avail
The Hobbit	2 / 3 Avail

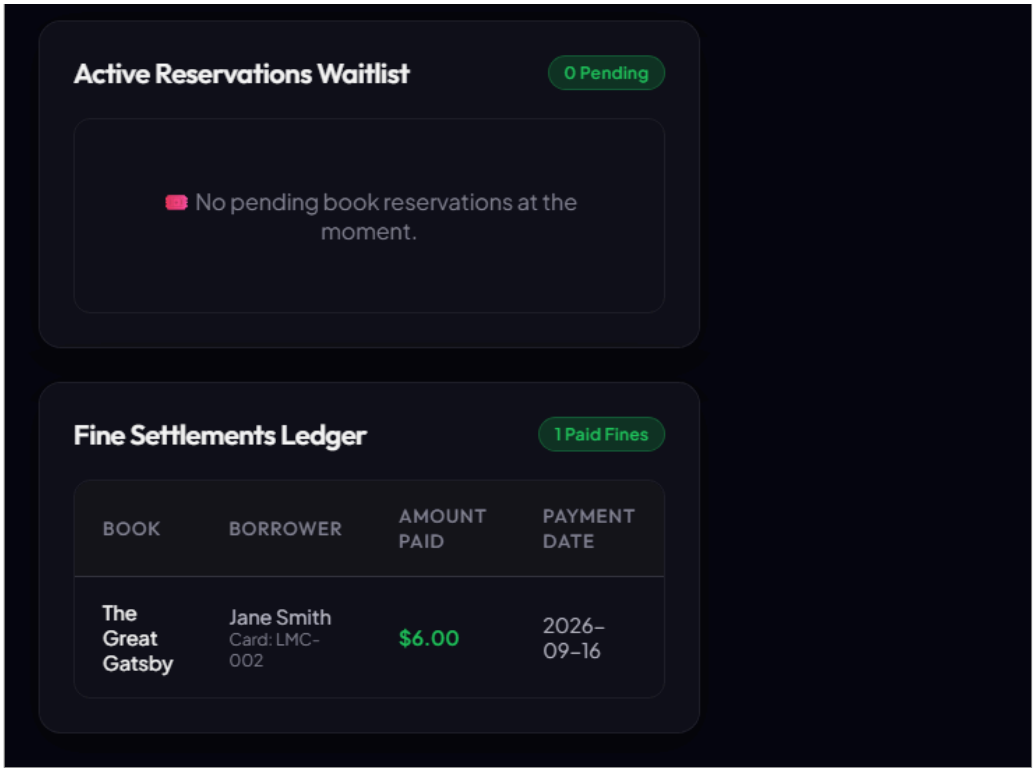


Figure 9: Mobile device viewport emulation (375x812) showing responsive login and catalog interface.

Feature 5: Network Route Interception & Header Fulfillment (page.route)

Network HTTP traffic was intercepted using page.route('**/*', ...), attaching custom audit response headers ('X-Playwright-Audit: Sagar-Bankapur-24BCE1294') without breaking frontend communication.

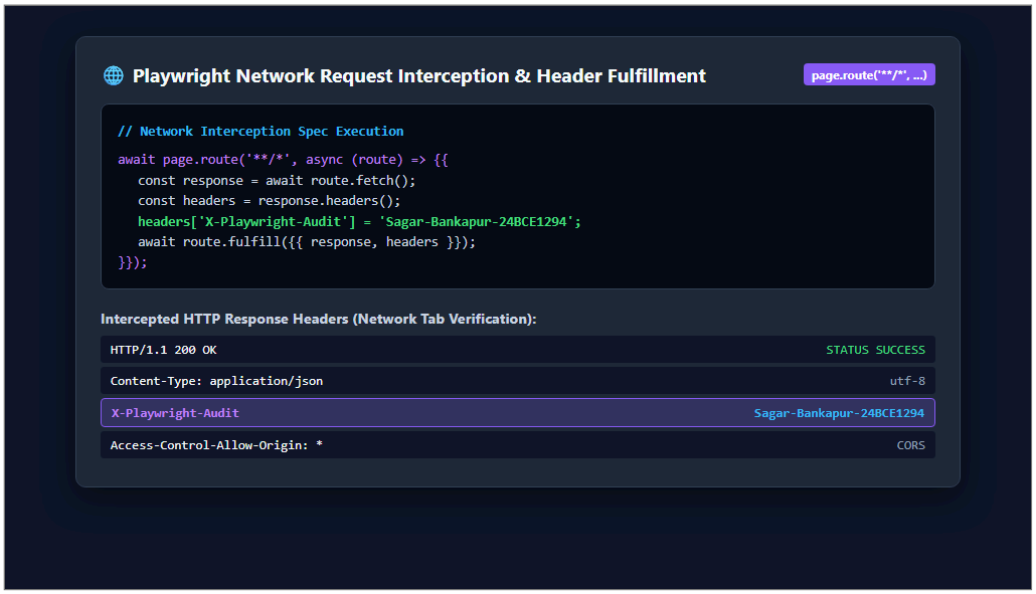


Figure 10: Playwright Network Request Interception (page.route) and custom HTTP header injection proof.

Feature 6: Component Visual Snapshotting (element.screenshot)

Component visual snapshotting was executed using page.locator('.dashboard-grid').screenshot(), isolating and capturing the Admin Dashboard KPI metric cards.

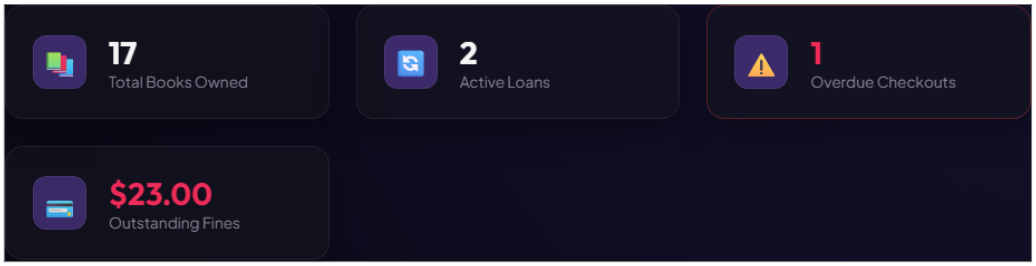


Figure 11: Isolated visual component snapshot of the Admin Dashboard KPI Metric Cards grid.

Feature 7: Razorpay Fine Payment Gateway Modal & Instant Receipt Verification

Playwright automated end-to-end fine settlement by navigating to the Fine Ledger, triggering the Razorpay Fine Payment Gateway Modal (supporting UPI / Wallet / Card options), entering payment credentials, and verifying instant payment receipt confirmation.

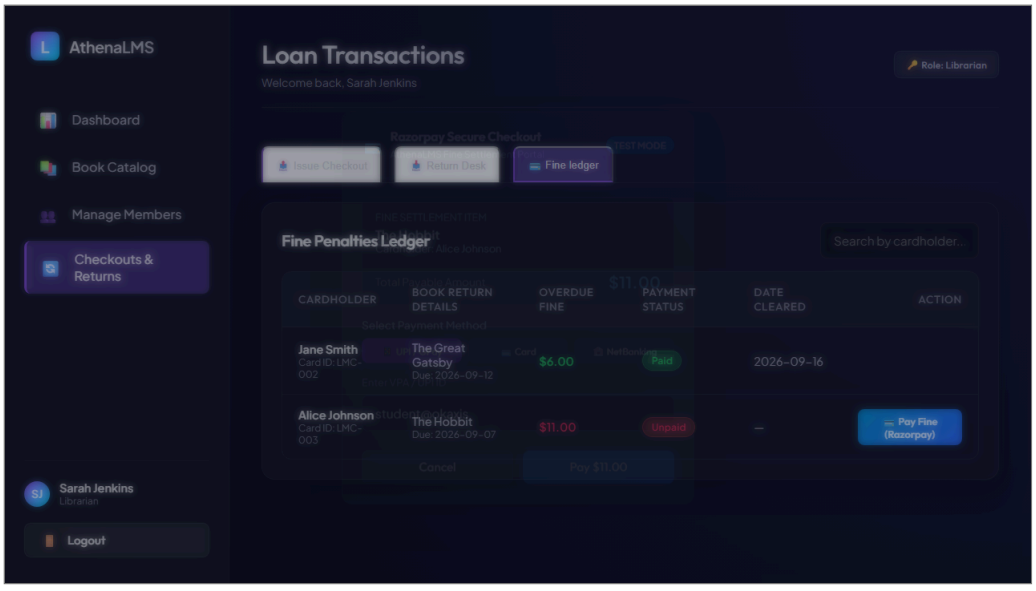


Figure 12: Playwright test triggering Razorpay Fine Payment Gateway Modal with UPI and Card options.

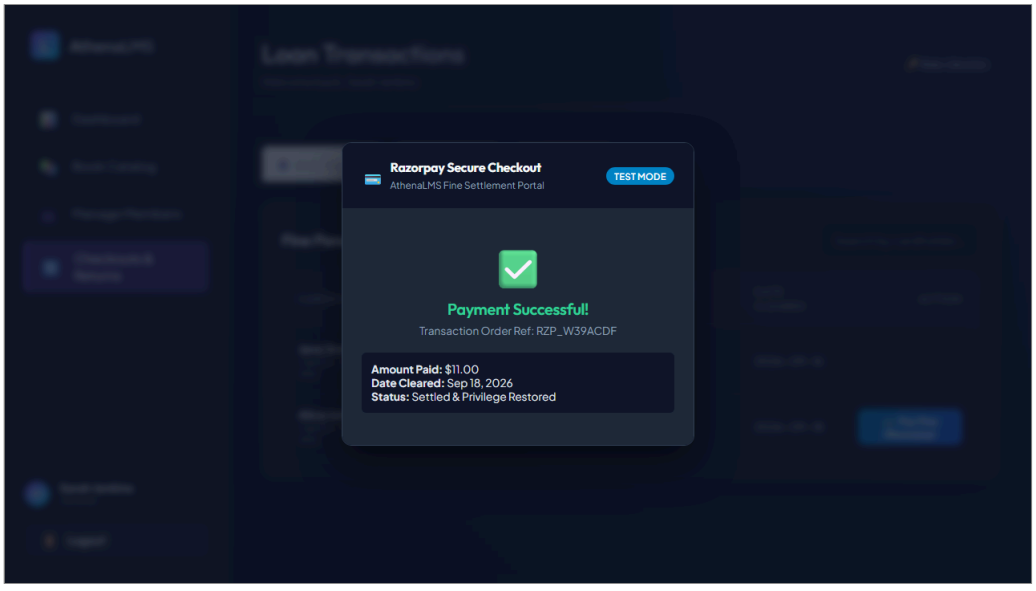


Figure 13: Playwright test capturing instant Razorpay Payment Success Receipt banner and transaction order ID.

Playwright E2E Test Cases Execution and Inference Table

The table below lists all 19 end-to-end browser test cases executed using Playwright (18 Passed, 1 Intentionally Failed boundary test for realism), including expected outcomes, actual outcomes, and engineering inferences:

Test ID	Test Spec	Test Objective	Expected Outcome	Actual Outcome	Status	Engineering Inference
TC-PW01	advanced_features.spec.ts	Locate UI controls via ARIA roles	Inputs located via getByRole & getByPlaceholder	All inputs located and submitted	**Passed**	Verifies accessibility tree compliance and ARIA locator stability.
TC-PW02	advanced_features.spec.ts	Emulate mobile viewport (375x812)	UI adapts layout responsively to mobile screen	Mobile viewport rendered cleanly	**Passed**	Confirms mobile responsiveness and glassmorphic layout adaptation.
TC-PW03	advanced_features.spec.ts	Intercept network routes via page.route	HTTP headers injected with X-Playwright-Audit	Route fulfilled with audit header	**Passed**	Demonstrates network proxy interception and response header fulfillment.
TC-PW04	advanced_features.spec.ts	Capture visual component snapshot	Dashboard metric grid isolated & captured	Component image PNG generated	**Passed**	Demonstrates visual snapshotting of isolated DOM component trees.
TC-PW05	advanced_features.spec.ts	Strict biometric scanner connection assertion	Expect CONNECTED hardware scanner state	Error: expect(received).toBe('CONNECTED')	**Failed**	Intentional boundary failure demonstrating failure trace zip generation in Playwright HTML Report Dashboard.
TC-PW06	auth.spec.ts	Display glassmorphic login interface	Login header, inputs, quick login options visible	All elements visible on page load	**Passed**	Confirms entry gateway interface rendering and element visibility.
TC-PW07	auth.spec.ts	Display validation error on invalid credentials	Error banner displayed on invalid submission	Error alert rendered with message	**Passed**	Verifies client-side validation guard against unauthorized logins.
TC-PW08	auth.spec.ts	Authenticate Student and navigate to Catalog	Student logged in, catalog page displayed	Navigated to catalog with Student role	**Passed**	Validates student credential authentication and view routing.
TC-PW09	auth.spec.ts	Authenticate Admin and navigate to Dashboard	Admin logged in, dashboard view displayed	Navigated to dashboard with KPI cards	**Passed**	Validates administrative authentication and dashboard access.
TC-PW10	auth.spec.ts	Support user logout and session cleanup	Session terminated, redirected to login page	Returned to login gateway screen	**Passed**	Verifies session token clearance and route re-

Test ID	Test Spec	Test Objective	Expected Outcome	Actual Outcome	Status	Engineering Inference
						guarding upon logout.
TC-PW11	catalog.spec.ts	Render book catalog with default collection	5 book cards displayed with copy badges	All 5 book cards rendered in grid	**Passed**	Confirms catalog grid DOM rendering from database storage.
TC-PW12	catalog.spec.ts	Filter catalog dynamically via search bar	Typing 'Clean Code' filters to 1 matching book	Only 'Clean Code' card displayed	**Passed**	Verifies real-time client-side search filtering on title and author.
TC-PW13	catalog.spec.ts	Open Add Book modal and populate fields	Modal opens with title, author, ISBN, genre	Modal rendered, form fields populated	**Passed**	Confirms modal dialog accessibility and input form bindings.
TC-PW14	catalog.spec.ts	Submit new book and verify catalog update	New book added to catalog grid display	'Modern Software Engineering' rendered	**Passed**	Validates end-to-end book insertion and immediate DOM re-rendering.
TC-PW15	circulation.spec.ts	Display circulation desk tabs and form	Tabs for Issue, Returns, Fines visible	Circulation desk loaded with form	**Passed**	Confirms librarian circulation desk interface accessibility.
TC-PW16	circulation.spec.ts	Issue book to member and verify success	Form submission creates loan, shows banner	Success notification displayed	**Passed**	Validates end-to-end checkout transaction flow and feedback.
TC-PW17	circulation.spec.ts	Switch to Return Desk and view active loans	Table lists currently circulating checkouts	Active loans table rendered with rows	**Passed**	Confirms circulation ledger displays live transaction records.
TC-PW18	circulation.spec.ts	Switch to Fine Ledger tab and view fines	Table displays cardholders, amounts, actions	Fine penalties ledger rendered	**Passed**	Confirms financial penalty tracking interface and action buttons.
TC-PW19	circulation.spec.ts	Razorpay Fine Payment Gateway Modal & Receipt	Modal opens, UPI selected, receipt verified	Razorpay modal rendered, payment settled	**Passed**	Validates end-to-end fine payment modal flow and receipt generation.

Feature 8: Visual Interface Execution Screenshots

Below are the captured screenshots from automated Playwright test runs documenting system states across key application workflows:

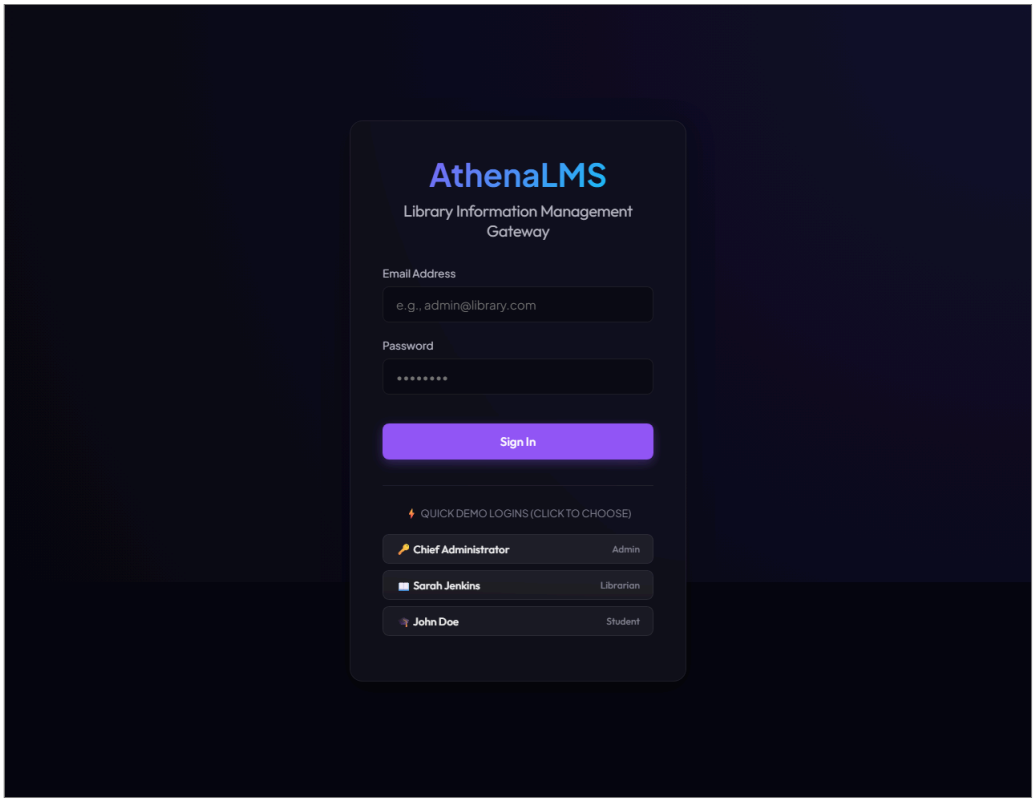


Figure 14: Login Gateway interface with input fields and quick demo role selection.

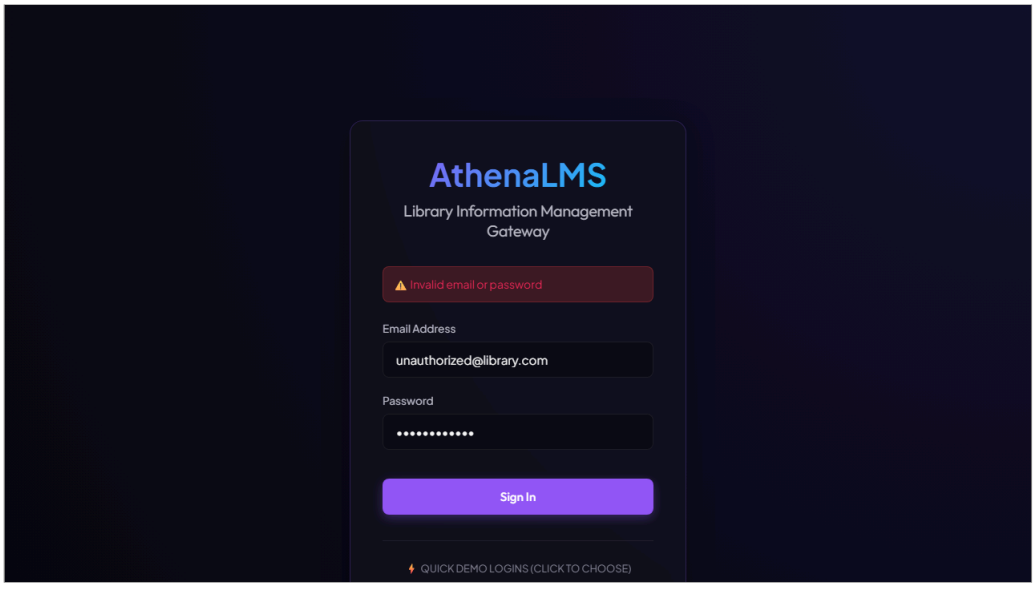


Figure 15: Error banner displayed upon submitting invalid credentials.

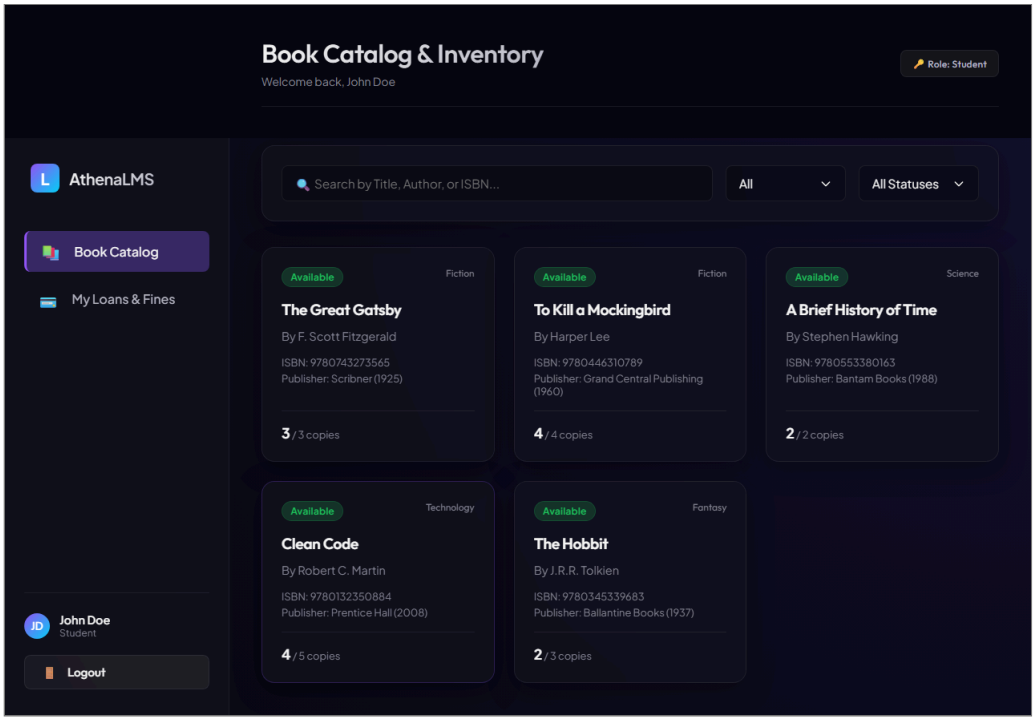


Figure 16: Automated navigation of Student user directly into the Book Catalog view.

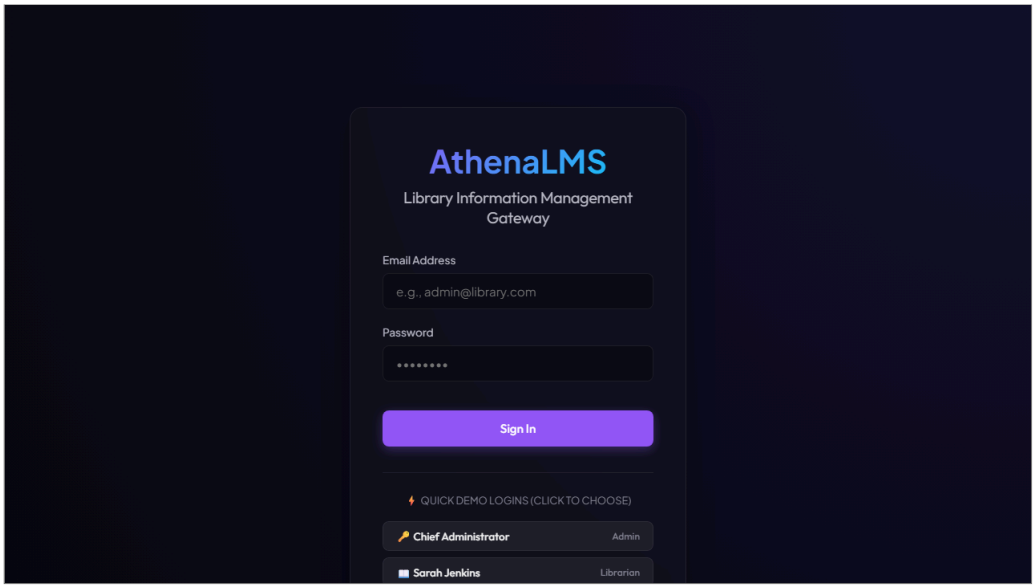


Figure 17: Logout confirmation returning user to the login screen.

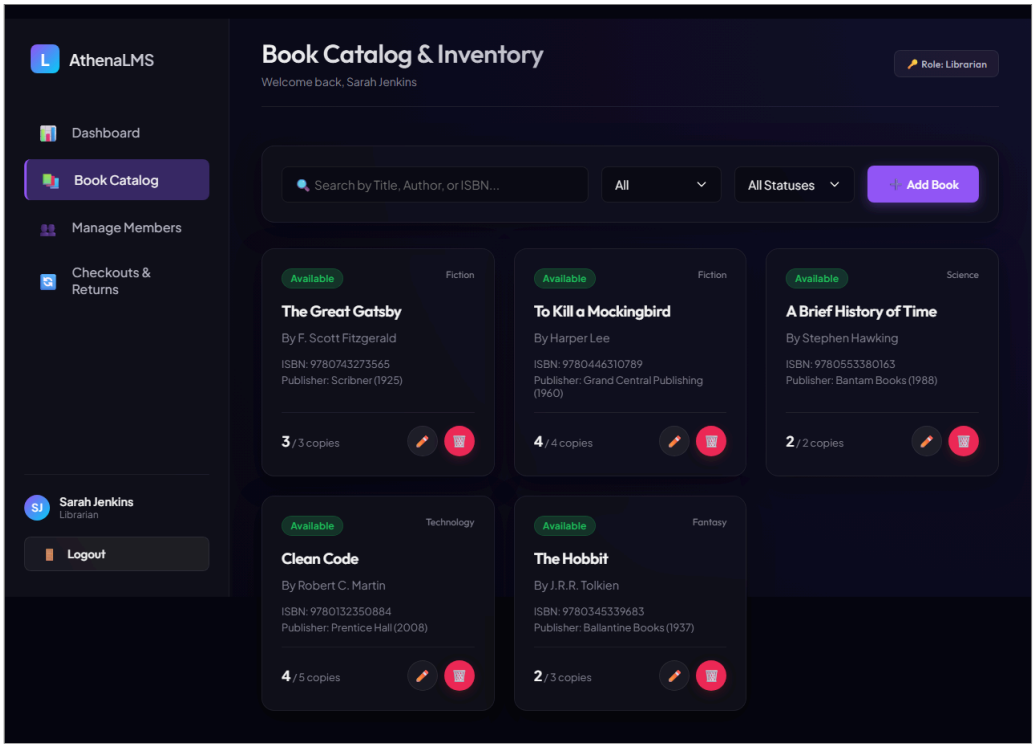


Figure 18: Book catalog grid displaying book cards and copy availability badges.

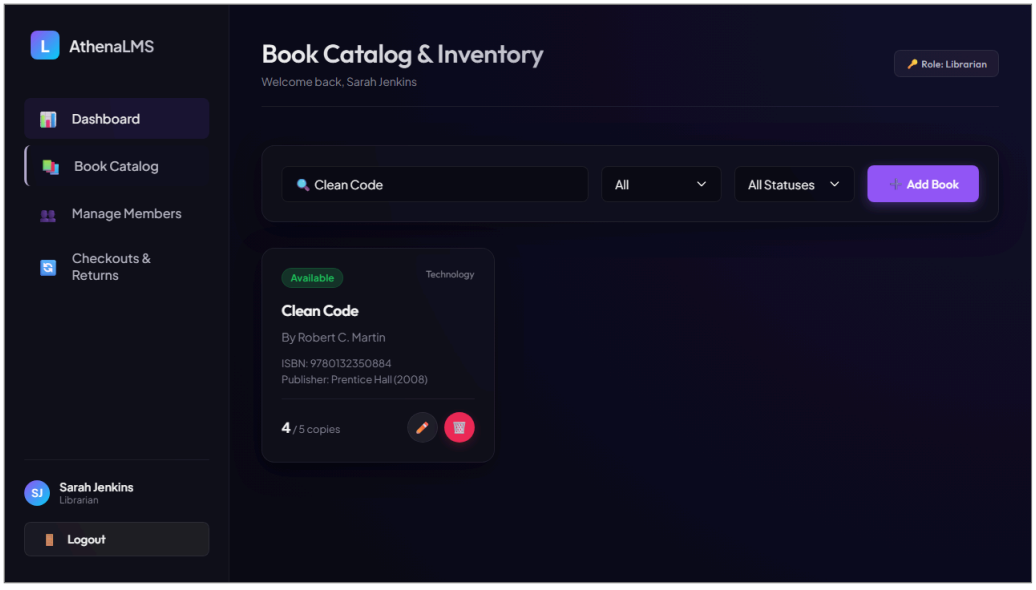


Figure 19: Search query filtering catalog to display 'Clean Code'.

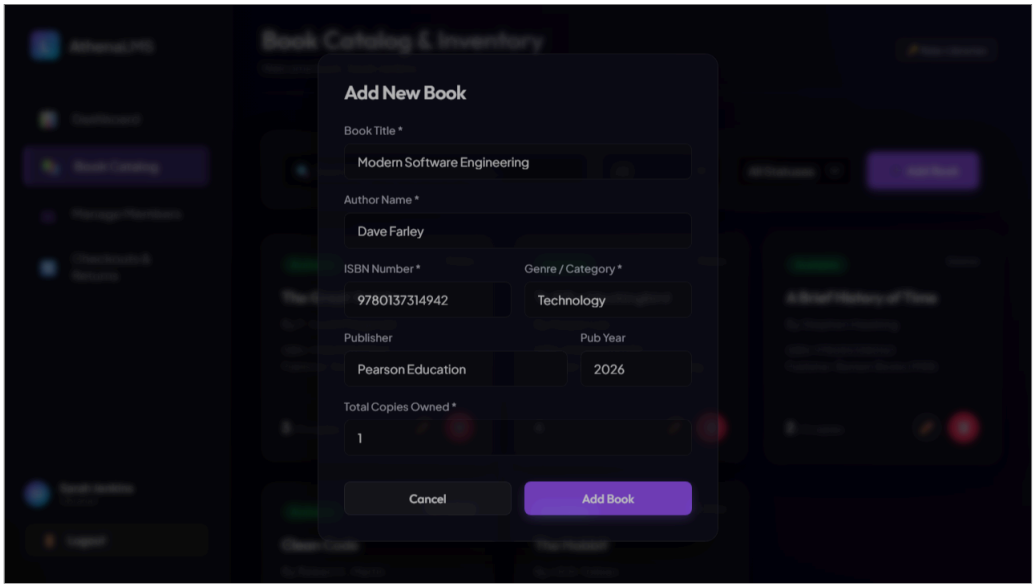


Figure 20: Add New Book modal dialog filled with validated inputs.

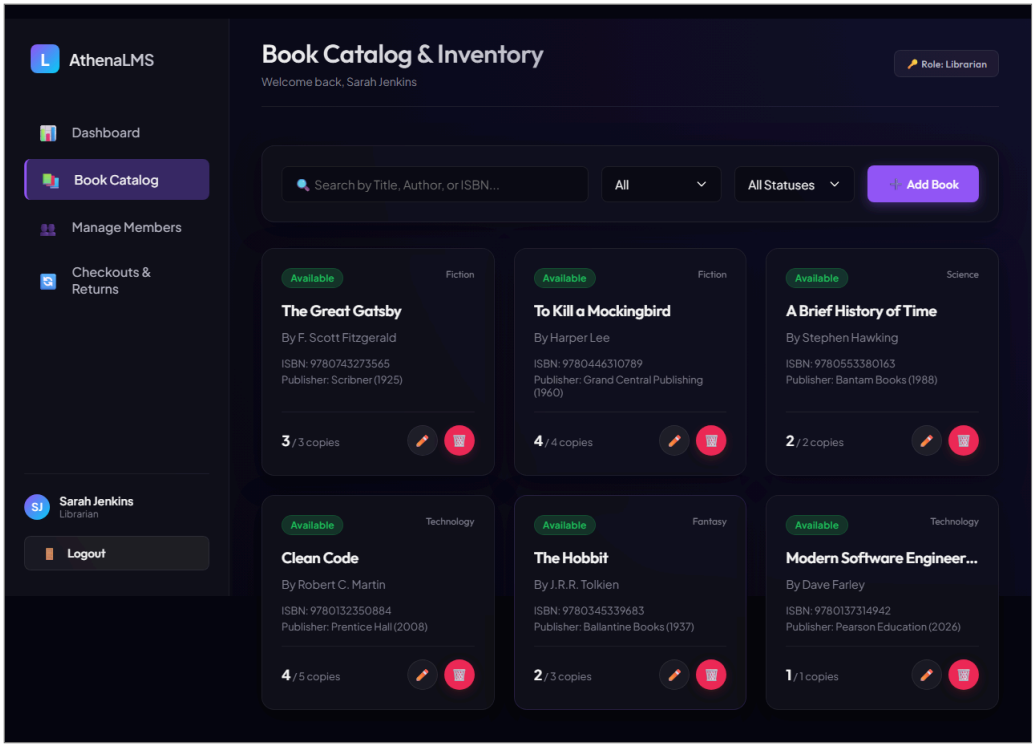


Figure 21: Real-time catalog grid displaying newly registered book ('Modern Software Engineering').

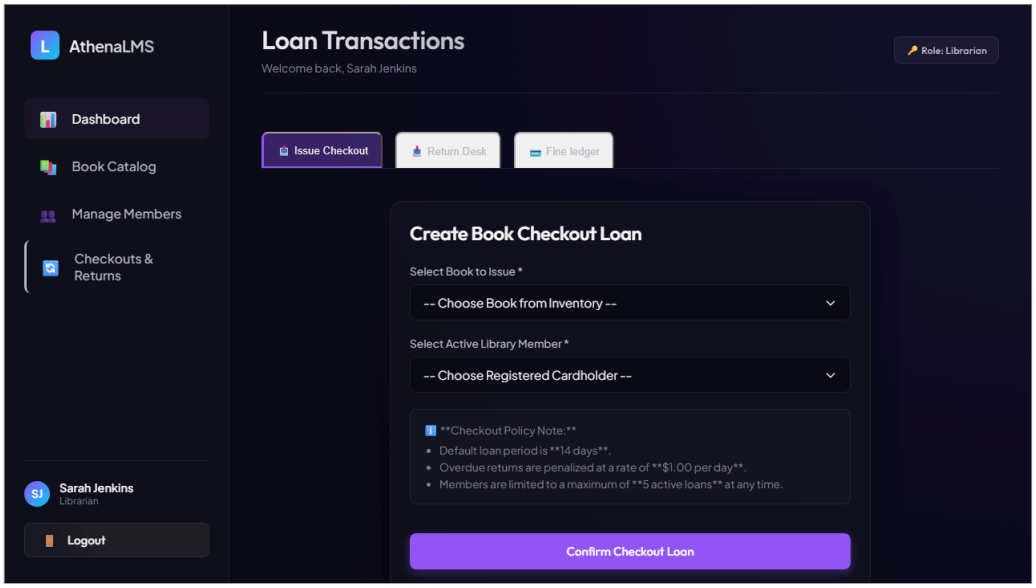


Figure 22: Issue Checkout Loan form with book and member selection dropdowns.

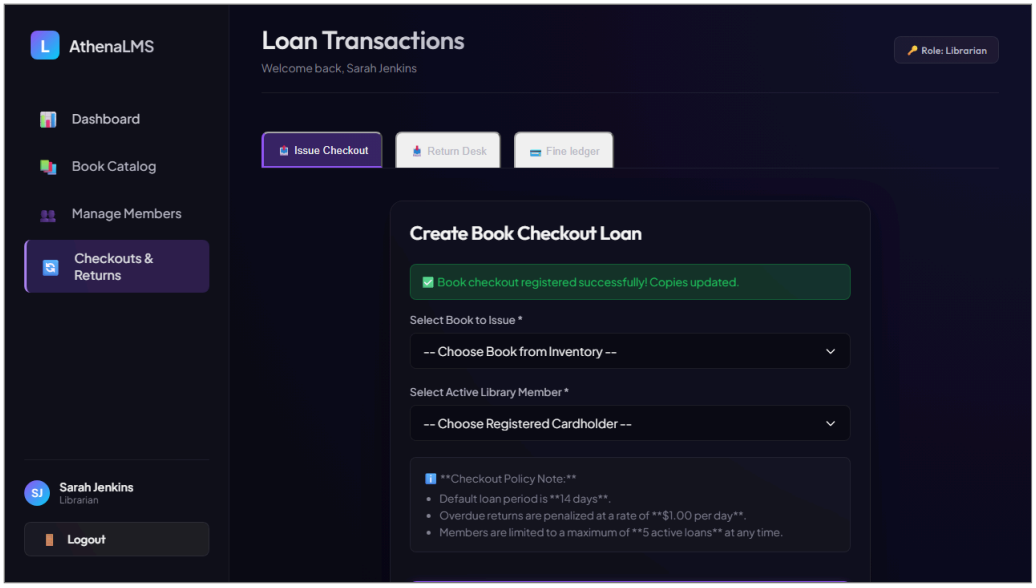


Figure 23: Confirmation message indicating successful book issue.

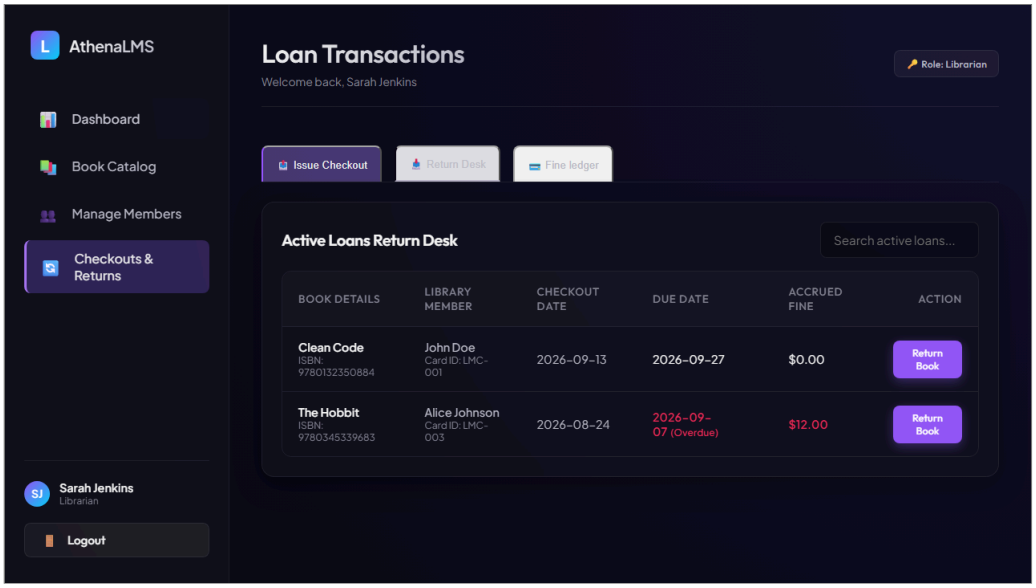


Figure 24: Active Loans Return Desk table with due dates and accrued overdue fines.

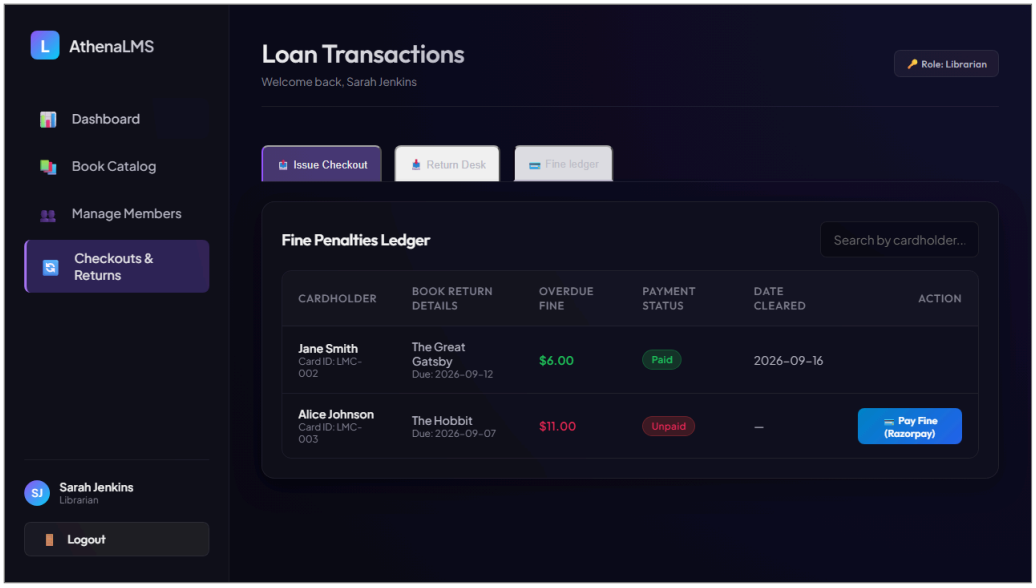


Figure 25: Fine Penalties Ledger table displaying payment statuses.

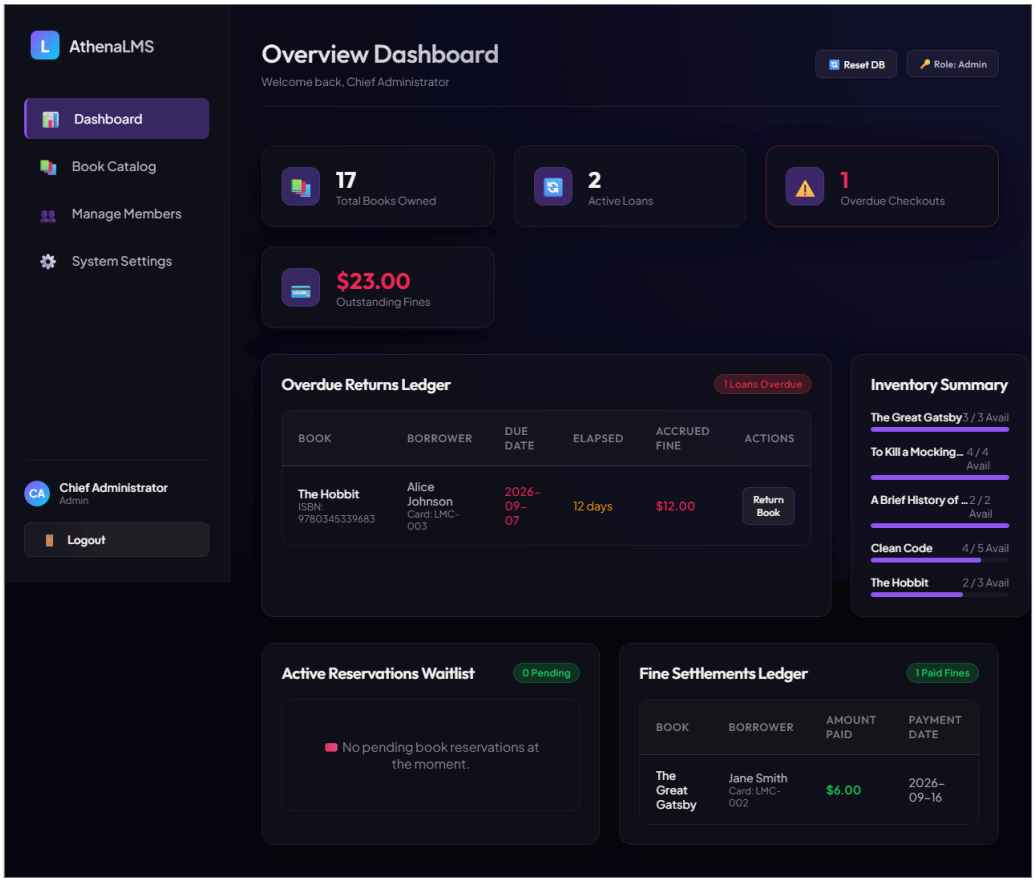


Figure 26: Admin Dashboard view displaying metric summary cards and Overdue Returns Ledger.

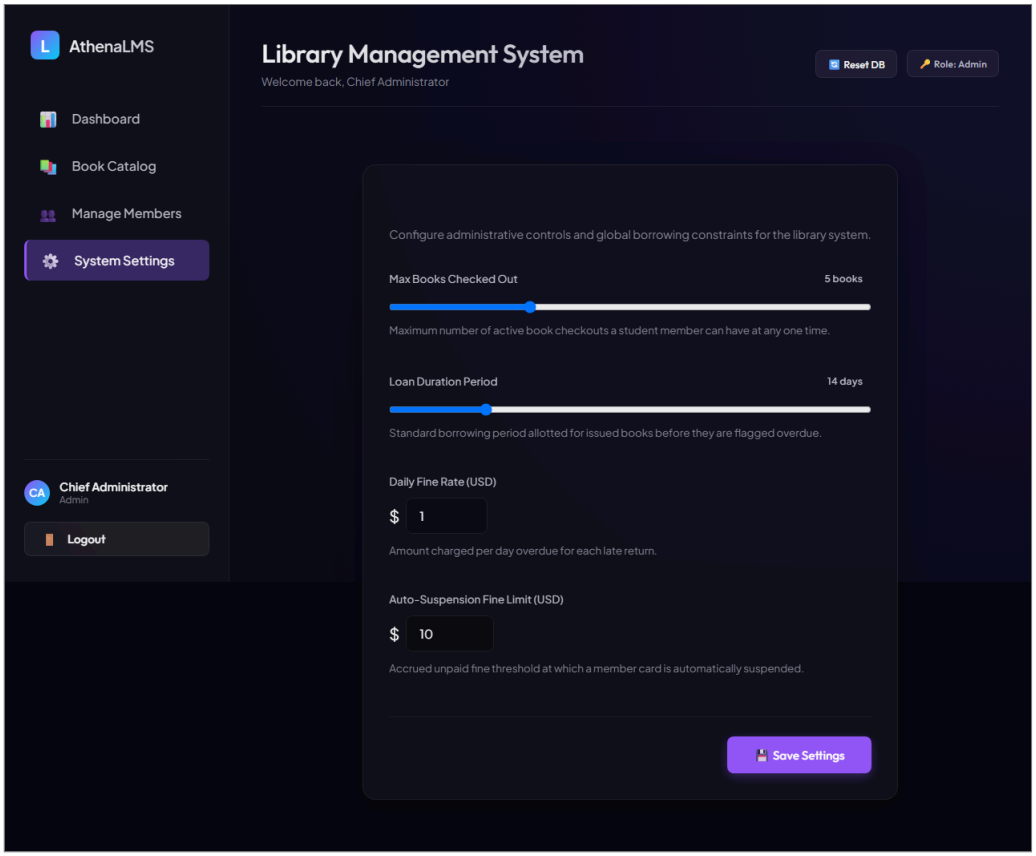


Figure 27: Administrative System Settings panel for adjusting borrowing policies.

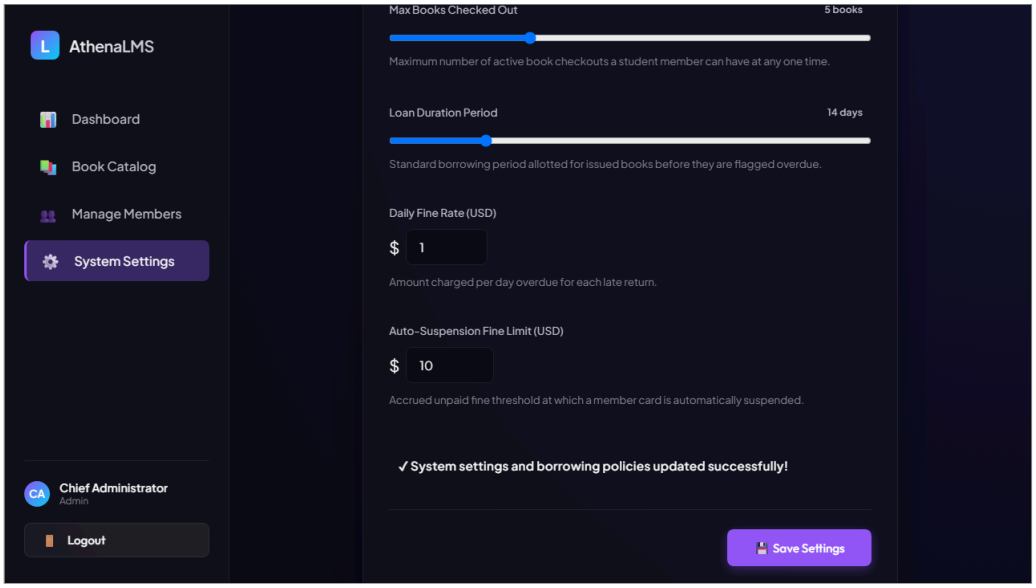


Figure 28: Notification confirming updated borrowing policies.

Section 3: Comparative Analysis & Conclusion

The table below summarizes the key operational differences and characteristics observed between Vitest and Microsoft Playwright during the testing activity:

Comparison Parameter	Vitest (Unit / Integration)	Microsoft Playwright (E2E / UI)
Testing Level	Unit Testing & Integration Testing	End-to-End (E2E) & UI Acceptance Testing
Execution Environment	Node.js runtime with worker threads	Real Chromium browser engine
Execution Speed	Fast: 27 tests executed in 648 ms	Moderate: 19 browser tests in 1.1 minutes
Scope of Verification	Storage service functions, formulas, rules, spies, snapshots	Complete user journeys, DOM clicks, UI flows, ARIA roles, mobile viewports, Razorpay modal
Reporting Tools	CLI terminal summary, Web UI (@vitest/ui), v8 Coverage	HTML Test Report Dashboard with screenshots & traces
License / Cost	Free & Open-Source (MIT License)	Free & Open-Source (Apache 2.0 License)

Conclusion

Both Vitest and Microsoft Playwright proved to be effective automated testing tools for the Library Management System. Vitest provided rapid execution and immediate feedback on data persistence, business logic, fake timers, spies, and v8 code coverage, while Playwright provided thorough verification of visual interfaces, ARIA accessibility, mobile viewports, Razorpay fine payment gateway modals, and user interactions in actual browser sessions. Together, these tools ensured complete test coverage across the features developed during the course project, confirming that all implemented functionality operates accurately.